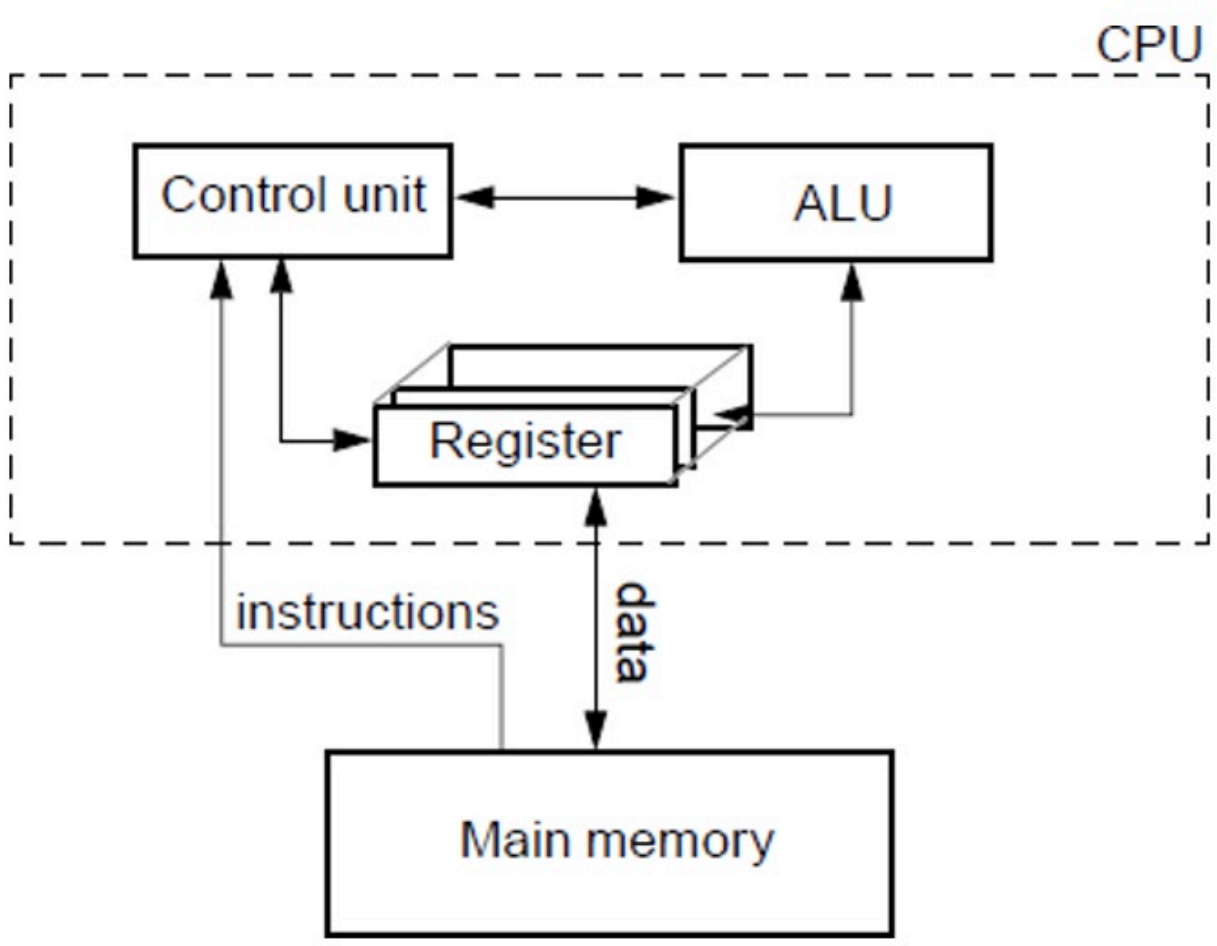
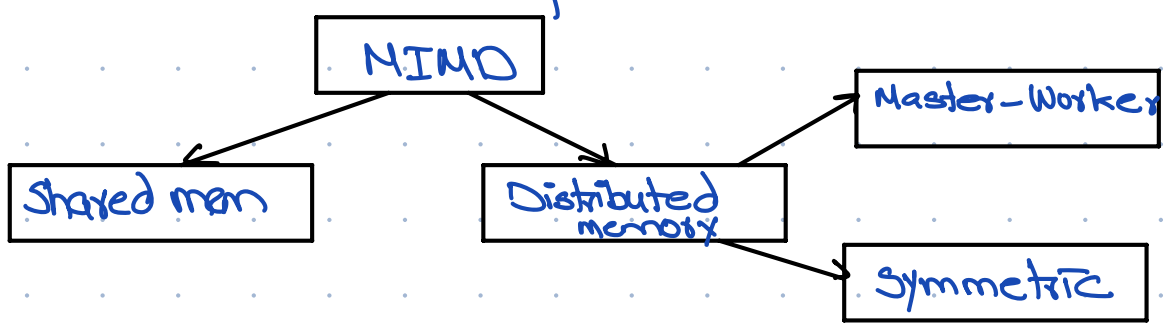
Uni-processors

- A computer with single chip/CPU
- Parallelism achieved through
  1. Instruction pipelines (ILP)
  2. Switch SIMD architectures (data level parallelism)

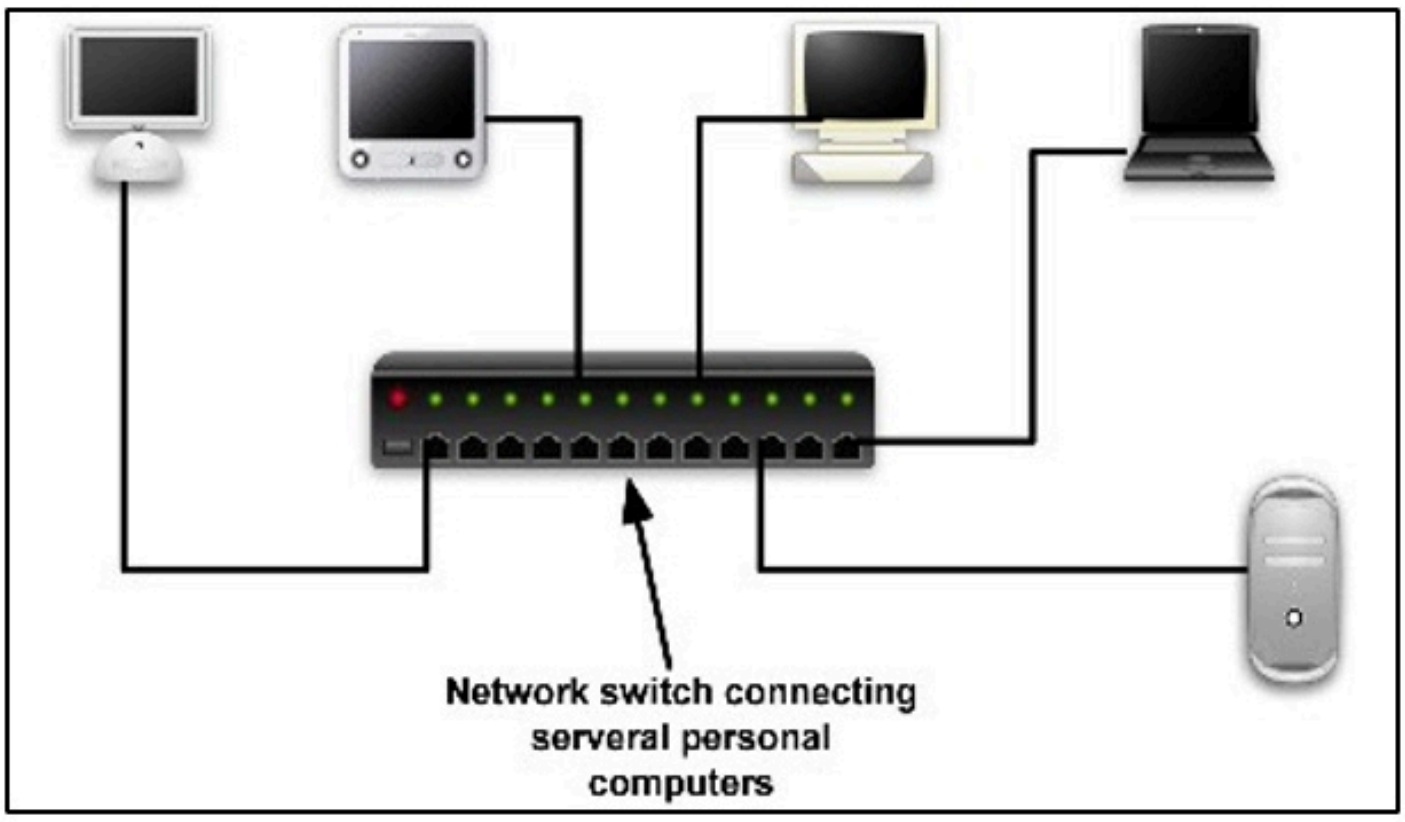


MIMD Architectures:

- Multiple instruction stream, multiple data stream systems.



- Multi-computer
  - Two or more independent computers (each with it's own local memory) interconnected via a network
  - Connect existing smaller computers to create a powerful system



↳ This requires hardware for interconnection & software to work with a variable no. of constituent computers.

- If many threads try to access memory, they compete for the shared bus. This creates a bottleneck where threads must wait for their turn.

- Multi-processor

  ↳ Two or processors with a common shared memory

  ↳ multi-processor microprocessor = multi-core

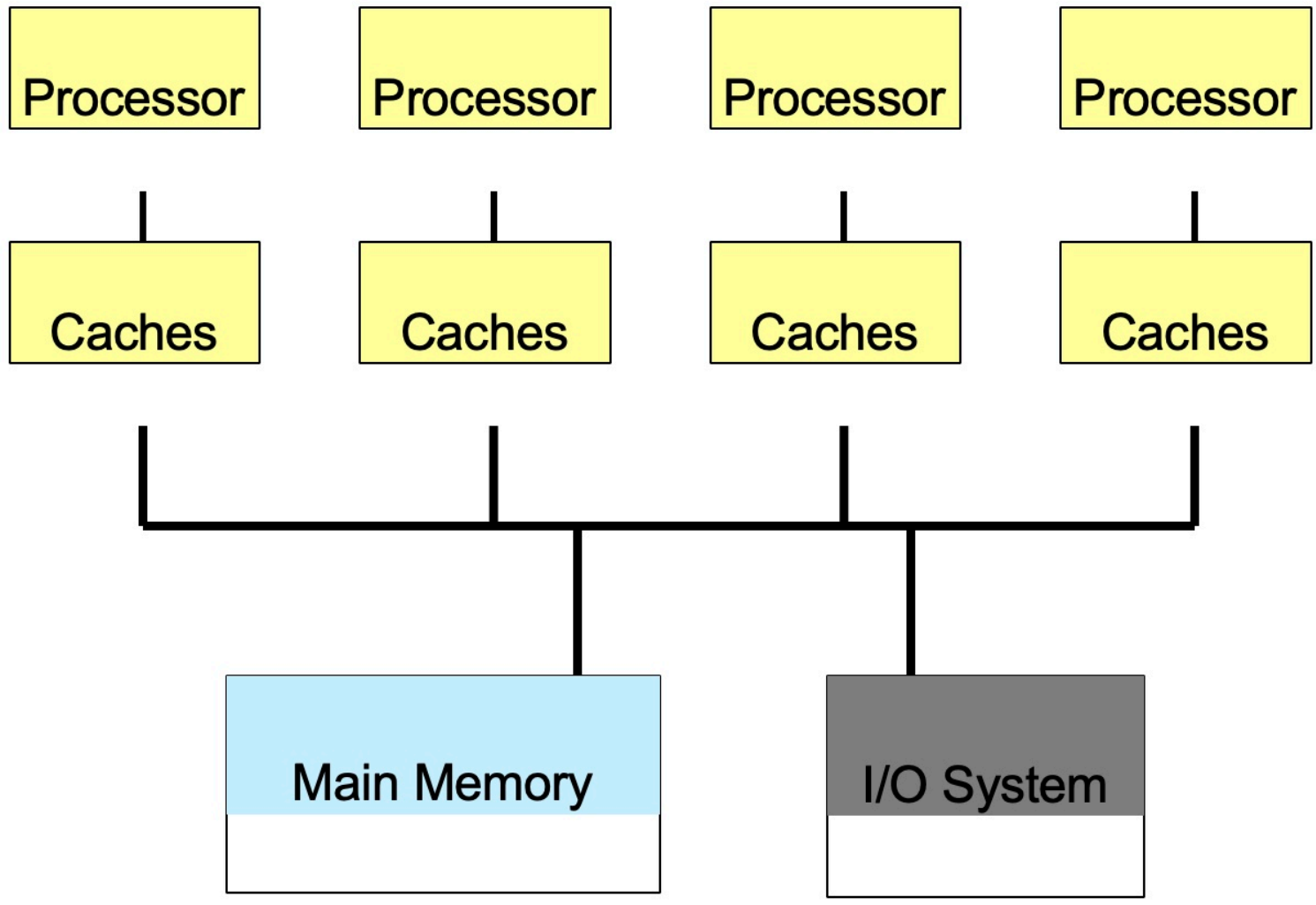
↳ Such systems are fault tolerant i.e if a computer in the network fails then other take up it's work

Classification of micro-processor

- This classification is based on the way their memory is organized.

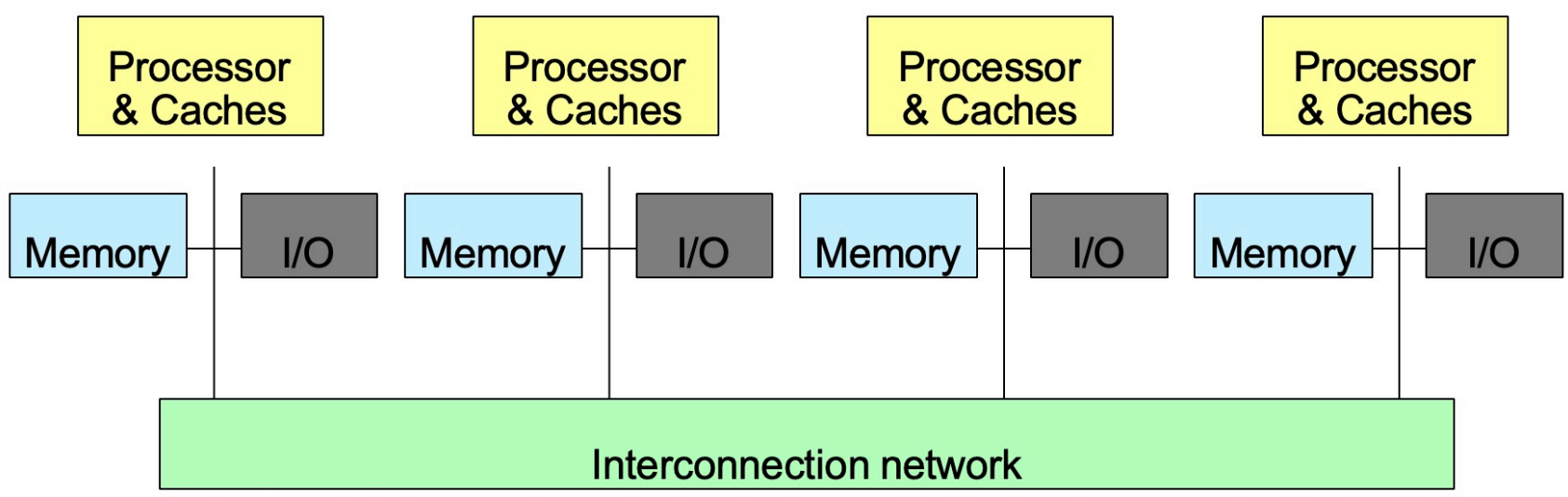
1. Tightly coupled micro-processor:

   ↳ Multi-processors connected to a single centralized memory



2. Loosely coupled multi-processor

- Memory is distributed among processors.
- Distributed memory multi-processors
  - Called cluster of computers
- If memory are strictly local, we need message-passing to communicate information from one processor to another.



Multi-processor properties

1. Controlled by a single user
2. Appear to users as a single system
3. Takes very little physical space

4. Improved power efficiency

5. Improved reliability
   - If a processor fails in a multi-processor with n processors then the system would continue providing service with n-1 processors.

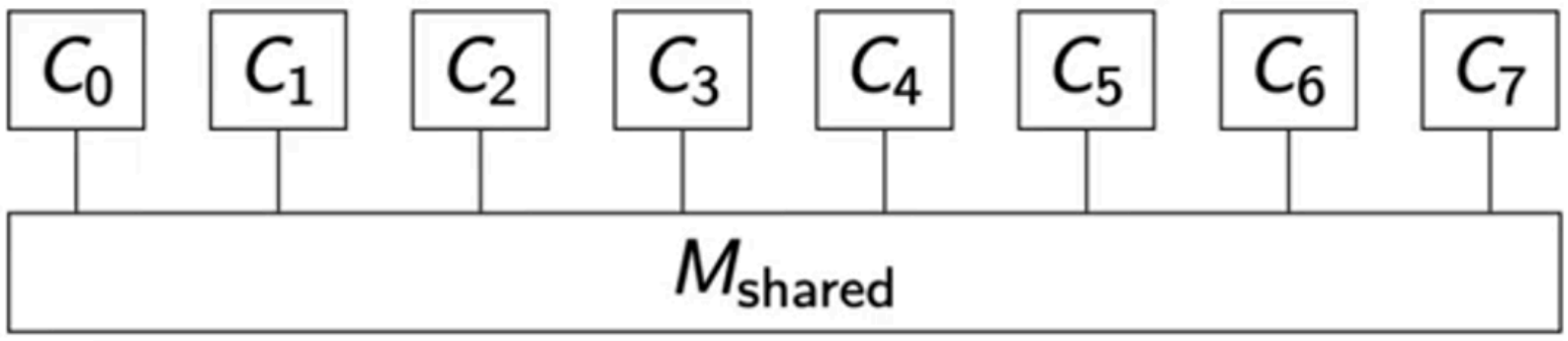
6. Improved performance
   - In one of two ways:
     a. Job (or Process) level parallelism
        - Running multiple independent jobs (programs or applications) simultaneously on multiple processors.
     b. Parallel processing programs
        - Running a single job on multiple processors

Micro-processor challenges

1. Learn to write parallel code
2. Computer designers must make hardware & software that makes it easier to write correct parallel programs.

MIMD in detail

1. Shared memory
   - All CPUs can access same memory
   - Simpler but is a bottleneck that limits scalability; can partition memory per CPU

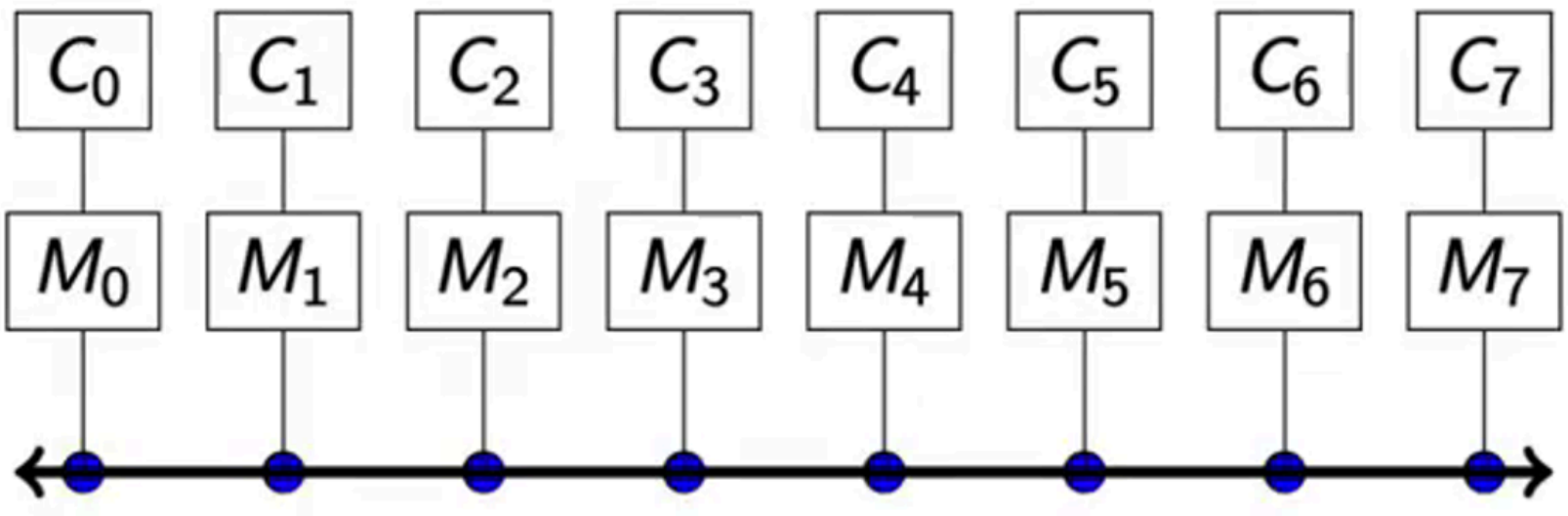


2. Distributed memory

↳ Each CPU has it's own local memory

↳ Symmetric – all cpus are equivalent; can do anything

↳ Master-Workers — Different cpus perform different tasks; In gpu system, GPU is a 'worker'.



Performance Metrics

- Speed Up:

↳ Captures the performance improvement of a parallel algorithm running on p processors compared to the best sequential algorithm on 1 processor.

$$S = \frac{t_1}{t_p}$$

where,

t_1 = Run time on 1 processor

t_2 = Run time on p processors.

Remarks:

↳ Normal range $[1, P]$

↳ $S = p$ called linear speedup — not possible

↳ t_p measured in "wall clock" time

↳ Notoriously hard to measure accurately

↳ Influenced by programmer, compiler, OS, load etc

↳ Must test under identical hardware & software, identical operations

↳ Use the fastest sequential algorithm available.

- Efficiency:

↳ Expresses how well a parallel algorithm makes use of the available resources.

$$E = \frac{S}{P}$$

$$= \frac{t_1}{P \cdot t_p}$$

Remarks:

↳ Range $[0, 1]$

↳ Linear speed up gives $E = 1$ (very rare)

↳ Always run-time overhead

↳ Communication overhead over processors

↳ Contention over shared memory

↳ Unbalanced workload — Idle CPUs